

게임 포트폴리오

모바일 3D 서브컬처



목차

1. 소개
2. 게임 소개
3. 사용한 툴
4. 게임 핵심 로직
5. 게임 최적화
6. 문제 및 해결

소개



프로젝트 개요

이 포트폴리오는 모바일 3D 서브컬처 게임입니다.



핵심 역량 및 생산성

- 과정에서 리소스 관리에 집중하여 최적화된 환경을 구축했습니다.
- 다수의 플러그인을 효과적으로 활용하여 개발 생산성을 극대화했습니다.

게임 소개



필드에 있는 몬스터를 제거합니다.

조작 방식

ATTACK: 일반공격 / BOOM: 전체공격 / DASH: 대쉬
및 저스트 회피



클리어 시 장비, 돈 및 스테이터스 포인트를
획득합니다.

게임 소개



스테이터스 배분

LEVEUP 버튼을 눌러 스테이터스를 배분합니다.



장비 장착

EQUIP에서 장비를 장착합니다.



장비 강화 및 합성

BAG에서 장비를 레벨업, 합성할 수 있습니다.

전투 진입

STAGE를 눌러 전투에 들어갑니다.



캐릭터 상호작용

중앙의 캐릭터를 드래그하고 눌러서 상호작용할 수 있습니다.

사용한 툴

Project Info

Unity 6000.0.67f1

Movement

- NavMesh AI

System

- **UniTask**
: 비동기 로직 최적화
- **UniRx**
: 데이터 스트림 및 이벤트 처리
- **Addressables**
: 메모리 및 에셋 관리
- **Google Cloud Storage**
: 리소스 다운로드

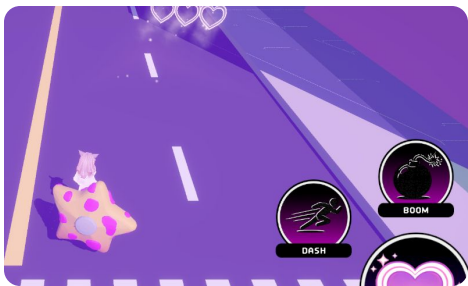
Optimization

- Lightmap
- Light Probe Group : 오브젝트 라이팅 최적화

게임 핵심 로직: 총알 및 이펙트 관리

사용 기술

- `UnityEngine.Pool.ObjectPool`
- `Addressables`
- `UniTask`



구현 상세

01

총알, 이펙트, 몬스터 등 빈번한 객체 생성을 위해 **ObjectPool** 시스템을 도입하여 GC 부하를 최소화했습니다

02

런타임 성능 확보를 위해 씬 로드 시점에 필요한 객체들을 **사전 생성**하여 사용합니다.

03

사용 완료된 객체는 즉시 풀에 **반환**하여 재사용성을 극대화했습니다

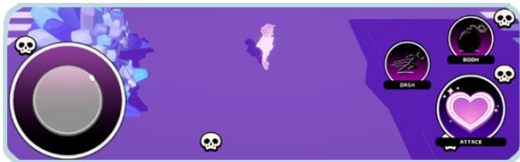
04

Addressables와 **UniTask**를 연동하여 리소스 로드 및 인스턴싱 과정을 효율적인 비동기 방식으로 처리합니다.

게임 핵심 로직: 키 조작

사용 기술

- UniRx
- System.IObservable



구현 상세

01

ReactiveProperty으로 버튼 눌렀을 때 상태를 알립니다.

02

IObservable<T>로 옵저버에게 상태 변화를 알립니다.

03

CompositDisposable로 사용하지 않게 된 옵저버들을 폐기(Dispose) 합니다.

게임 핵심 로직: 스테이터스 변화

사용 기술

- `UniRx.Subject<T>`
- `System.IObservable`



구현 상세

01

최종 스테이터스 변화를 `Subject`와 `IObservable<T>`로 알립니다.

02

최종 스테이터스는 캐릭터 스테이터스와 장비의 추가 스테이터스의 합산입니다.

03

합산 스테이터스를 `Subject`의 `.Next`로 알립니다.

04

`CompositDisposable`로 사용하지 않게 된 옵저버들을 폐기(`Dispose`) 합니다.

게임 핵심 로직: 저스트 회피

사용 기술

- UniTask
- DoTween
- UniRx.MessageBroker
- Post Processing



구현 상세

01

몬스터 공격 직전 회피 시 저스트 회피 발동 및 몬스터 감속

02

DoTween으로 캐릭터 Scale/Position 조절 및 UniTask 연출 대기

03

TimeScale 조정을 통한 전체적인 슬로우 모션 구현

04

캐릭터 관련 오브젝트는 TimeScale의 영향을 받지 않도록 설정

05

MessageBroker를 이용해 Global Volume에 회피 이벤트 전달

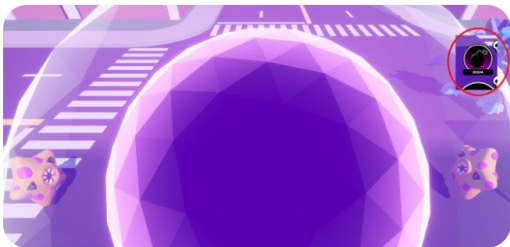
06

Post Processing(Vignette, Chromatic Aberration 등)으로 시각 효과 극대화

게임 핵심 로직: 전체 공격 및 견인

사용 기술

- NavMeshAgent
- Dotween



구현 상세

01

전체공격 사용 시 주변 몬스터를 한 곳으로 모음

02

이동 전 NavMeshAgent 정지 및 Dotween을 이용한 위치 이동

03

콜백으로 현재 위치를 업데이트하며 NavMesh.SamplePosition으로 유효 위치 확인

04

유효 위치에 Warp를 호출한 후 NavMeshAgent를 다시 활성화

게임 핵심 로직: 상태 이상 체크

사용 기술

- `UniRx.Subject<T>`
- `System.IObservable`
- `UnityEngine.Pool.ObjectPool`



구현 상세

01

특정 무기 타격 시 특수 효과(감속, 지속 데미지, 기절) 부여

02

상태 이상 시 효과 아이콘, 캐릭터 색상 변경 및 이펙트 생성

03

상태 이상 이펙트는 `ObjectPool`을 사용하여 메모리 관리 최적화

04

최근 상태 우선 표시 및 이전 이펙트 풀 보관, 색상 동적 변경

05

다중 상태 적용 관리를 위해 `Bitmask` 방식으로 데이터 전달

06

`Subject`와 `IObservable`을 활용한 리액티브한 값 변화 감지

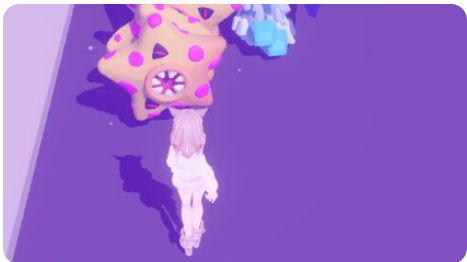
07

`SkinnedMeshRenderer.material`을 통한 실시간 캐릭터 색상 제어

게임 핵심 로직: 에임 보조 시스템

사용 기술

- `Physics.OverlapSphereNonAlloc`
- `LayerMask Targeting`



구현 상세

01

캐릭터의 공격 시 시야 내 가장 가까운 몬스터를 자동으로 타겟팅하여 공격 방향 보조

02

정면 시야각 내의 몬스터만 필터링하여 측면/후면 몬스터 오타겟팅 방지

03

`OverlapSphereNonAlloc`를 활용해 가비지 컬렉션(GC) 발생을 최소화하며 타겟 감지 구현

게임 핵심 로직

기타 게임 로직

1. 리소스 해제 및 종료 관리

- `CancellationTokenSource`를 통해 씬 종료 시 관련 오브젝트 일괄 종료

2. 인터페이스 기반 결합도 감소

- `Interface` 상속 및 전달을 통해 필요한 기능만 노출
- 예: `ITargetable`을 통한 몬스터의 캐릭터 위치 정보 획득

3. 데이터 전달 최적화

- `MessageBroker/Subject` 사용 시 `Struct`나 값 타입만 전달하여 객체 생성 최소화

게임 최적화

Statistics

Audio:

Level: -74.8 dB (MUTED) DSP load: 0.1%
Clipping: 0.0% Stream load: 0.0%

Graphics:

76.6 FPS (13.0ms)
CPU: main 13.1ms render thread 1.4ms
Batches: 140 Saved by batching: 0
Tris: 169.6k Verts: 254.8k
Screen: 2160x1080 - 26.7 MB
SetPass calls: 45 Shadow casters: 23
Visible skinned meshes: 22
Animation components playing: 0
Animator components playing: 12

1. 라이팅 및 렌더링 최적화

정적 환경의 실시간 라이팅을 줄이고 라이트맵으로 전환하여 발열 및 부하 완화

- Batches: 317 → 144 수준으로 대폭 감소

2. 라이트맵 용량 대응 및 전송 최적화

- Addressables 기반 원격 다운로드 및 1회성 캐시 적용 (100MB+ 대응)
- Lightmap Resolution 하향 (40 → 30)을 통한 추가 용량 확보

3. 동적 오브젝트 품질 보완

- Light Probe Group을 적용하여 캐릭터 및 동적 오브젝트에 환경광 반영
- *Adaptive Probe Volume*은 *Unity Toon Shader* 호환성 이슈로 제외

4. 캐릭터 셰이더 및 패스 최적화

- Outline 패스 제거를 통한 SetPass Calls 병목 현상 해결
- 최종 결과: Batches 140 / SetPass Calls 45 (Shadow Casters 199 → 23)

게임 최적화

배칭 및 인스턴싱

- 몬스터와 맵 오브젝트는 **SRP Batcher**로 묶어 최적화
- **Enable GPU Instancing** 옵션을 활성화하여 성능 개선

에셋 관리 및 기능 활용

- **URP, Addressables** 등 유니티 제공 기능 적극 활용
- 이미지를 **Sprite Atlas**로 효율적으로 관리

메모리 최적화

- 오브젝트 풀링을 사용하여 런타임 부하 방지

문제 및 해결

문제

- 어두운 곳에서 캐릭터 부츠의 빛이 튀고, 밝은 곳에서 그림자가 진하게 적용됨.
- Light Probe Group 적용 후 캐릭터가 어둡게 나옴

원인

- 맵의 단차가 있는 곳에서 빛이 과하게 튜는 현상 발견
- 바닥 밑의 Light Probe Group의 프로브가 검은색 나옴

해결

- 맵의 Light Probe Group의 높이값을 동일하게 셋팅
- 모든 Light Probe Group 높이값을 위로 올림.

문제 및 해결

문제

- `UniTask` 취소 처리가 제대로 실행 되지 않아, 코드가 계속 실행되어 에러 발생

원인

- `OperationCanceledException`는 사용하고 있으나 예외를 던지지 않고 있음.

해결

- `OperationCanceledException` 에서 `throw`를 호출해 예외를 상위로 보냄.
- `CancellationTokenSource`는 씬에서만 가지고 있고 취소하면 한번에 취소시켜 관리를 편하게 함.

문제 및 해결

문제

- 캐릭터 에임이 보정되지 않아 총알이 몬스터에게 맞지 않음.

원인

- 편의를 위해 보정 기능 구현 필요.

해결

- 에임 보정 기능 구현.
- 범위를 최소한으로 하여 최적화.

감사합니다

정우철

연락처: 010-3569-2410

Email: chockbal@gmail.com

포트폴리오 및 블로그

네이버 블로그: <https://blog.naver.com/mimy3131>

영상 링크

YouTube: <https://youtu.be/N4EuXiwo5aY>